RTLCraft: A White-Box Framework for LLM-Driven RTL Design, Verification, and Co-Evolution

Ji Zhuang[1], Yanqiu Cao[1], Fan Yang[1]
[1]State Key Laboratory of Integrated Circuits and Systems,
College of Integrated Circuits and Micro-Nano Electronics, Fudan University
yangfan@fudan.edu.cn

Abstract

Existing large language model (LLM) approaches for hardware design generate RTL as opaque text strings, making functional validation and performance optimization a manual, black-box ordeal. In this paper, we present **RTLCraft**, an open-source framework that turns RTL design into a *white-box*, programmatically manipulable object. By representing hardware as a Python-embedded abstract syntax tree (AST), RTLCraft provides a structured interface through which an LLM agent—such as an in-context coding assistant (e.g., Kimi Code, Claude Code)—can **read**, **modify**, and **optimize** circuit descriptions at the source level. The framework closes three critical feedback loops: (1) a **functional self-consistency loop** via a Python-native UVM verification stack (pyUVM) that runs against a cycle-accurate AST simulator, (2) a **performance self-consistency loop** via AST-level PPA estimation and Berkeley ABC synthesis, which returns area/delay metrics back to the agent, and (3) an **RTL-tool co-evolution loop** in which the framework itself evolves alongside the designs it produces—enabling an agent to not only refine a circuit but also grow reusable libraries, protocol VIPs, and even processor/NPU compilers from the same substrate. We demonstrate these capabilities through LLM-in-the-loop examples ranging from an 8-bit counter to a pipelined ALU and protocol VIP, showing iterative refinement from buggy initial specifications to verified, synthesis-optimized implementations in under three agent–environment rounds, with per-iteration latency under 10 seconds.

1 Introduction

Register-transfer level (RTL) design remains a major bottleneck in digital hardware development. The conventional flow—manually authoring Verilog, writing testbenches in SystemVerilog/UVM, and iterating with logic synthesis—is labor-intensive and slow. Recent advances in large language models (LLMs) have sparked interest in automating RTL generation, yet most existing approaches treat the problem as a *text-to-text* translation task: an LLM receives a natural-language specification and emits a raw Verilog string. This black-box paradigm suffers from three fundamental limitations:

1. **Opacity:** Once generated, the Verilog text is difficult for the LLM to reason about or surgically edit. Fixing a bug or resizing a datapath usually requires regenerating the entire module.

2. **Weak validation loops:** Because the generated artifact is plain text, integrating it with simulators, verifiers, and synthesizers requires fragile parsing and ad-hoc scripting. There is no turnkey path from a failed UVM test or a poor post-synthesis area report back to the LLM in a structured form.

3. **Loss of semantic structure:** Verilog syntax is low-level and noisy. An LLM operating on raw text must simultaneously reason about clock-domain crossing, width matching, and operator precedence, which dramatically increases error rates.

1.1 Our Approach: White-Box LLM-Driven RTL

We propose **RTLCraft**, a unified Python framework that elevates RTL from an opaque text format to a **structured, white-box object model**. In RTLCraft, hardware is described via a Python API that constructs a typed AST. This design makes the circuit amenable to programmatic inspection and mutation: an LLM agent (or any Python script) can import RTLCraft, traverse the AST, modify parameters, swap arithmetic implementations, and re-synthesize without ever touching a line of Verilog text.

The central workflow enabled by RTLCraft is an iterative, closed-loop conversation between an LLM agent and the hardware design environment:

$$\textit{Spec} \rightarrow \textit{Python RTL-AST} \rightarrow \begin{array}{c}\textbf{pyUVM}\ \text{(functional)}\\ \textbf{PPA+ABC}\ \text{(performance)}\end{array} \rightarrow \textit{Structured Feedback} \rightarrow \textit{Agent Patch}$$

Because the simulator, the UVM testbench, and the PPA estimator are all native Python, feedback (e.g., a failing VCD trace or a synthesis QoR JSON) can be injected directly into the LLM's prompt or tool-use context. The agent therefore operates not as a text generator, but as a **design optimizer** that manipulates a white-box model.

1.2 Contributions

Our contributions are:

- **A programmatically manipulable RTL object model.** We introduce a Python-based RTL API and AST representation in which modules, signals, and control-flow constructs are first-class objects. This enables an LLM agent to perform surgical edits (e.g., changing an adder style or inserting a pipeline stage) via standard Python constructs.
- **Functional self-consistency via Python-native UVM.** We build **pyUVM**, a full UVM-like framework that executes in Python against RTLCraft's internal simulator. Failed tests, coverage reports, and waveforms can be fed back to the agent as structured context, enabling automated bug diagnosis and repair.
- **Performance self-consistency via AST-level PPA and ABC synthesis.** We provide a static PPA analyzer (depth, area, toggle power) and an ABC-based synthesis backend. The agent receives millisecond-scale PPA estimates for rapid exploration and gold-standard post-synthesis QoR for final convergence.
- **RTL-tool co-evolution: design-driven library growth.** We demonstrate that the same AST substrate used to generate datapaths also supports the agent-driven generation of protocol VIPs (APB, AXI4-Lite), UVM testbenches, cocotb tests, and even processor/NPU compilers—meaning the framework grows alongside the designs it produces.
- **End-to-end LLM-in-the-loop demonstrations.** We show how an agent, equipped with RTLCraft, iteratively generates and refines designs across complexity levels—from an 8-bit counter (where the agent catches a missing enable guard) to a pipelined ALU and APB protocol VIP—converging to verified, synthesis-optimized output in under 10 seconds per iteration.

2 Related Work

LLMs for Hardware Design. VeriGen[1], Chip-Chat[2], ChipNeMo[?], and VerilogEval[?] demonstrate that LLMs can produce syntactically correct Verilog. More recently, automated RTL generation from natural-language specs has been explored[?, ?]. However, these systems output raw text strings. Consequently, the LLM cannot easily inspect or modify its own output, and integration with downstream EDA tools requires external, hand-written glue scripts. RTLCraft differs fundamentally by making the RTL artifact a structured Python object that the agent can manipulate programmatically.

Python-Based HDLs. MyHDL[3] and Amaranth[4] provide powerful Python abstractions for hardware. Chisel[?] elevates hardware construction to a Scala-embedded DSL with sophisticated type inference. Yet none of these expose an interface specifically intended for automated agent manipulation, nor do they ship with an integrated UVM verification loop or ABC synthesis backend. RTLCraft is designed from the ground up as an *agent-facing* framework.

Verification Frameworks. cocotb[5] enables Python testbenches but still relies on an external SystemVerilog simulator. SystemVerilog UVM is the industry standard[6], but its steep learning curve and lack of Python introspection make it unsuitable for LLM-driven loops. RTLCraft's pyUVM executes entirely in Python, allowing the agent to query simulation state and coverage bins directly.

High-Level Synthesis. LegUp[7] and Bambu[8] compile C/C++ to RTL. These tools raise the abstraction level but sacrifice fine-grained control. RTLCraft operates at the RTL level, giving the agent direct, white-box access to datapath microarchitecture.

Self-Consistent Reasoning. Wang et al.[?] showed that LLM self-consistency improves reasoning quality. We extend this principle to hardware design by defining two complementary consistency loops—functional (via pyUVM) and performance (via PPA + ABC)—that together converge toward a correct and optimized design.

3 RTLCraft Framework

Figure 1 shows the RTLCraft architecture, explicitly highlighting the feedback paths that enable an LLM agent to operate in a closed loop.

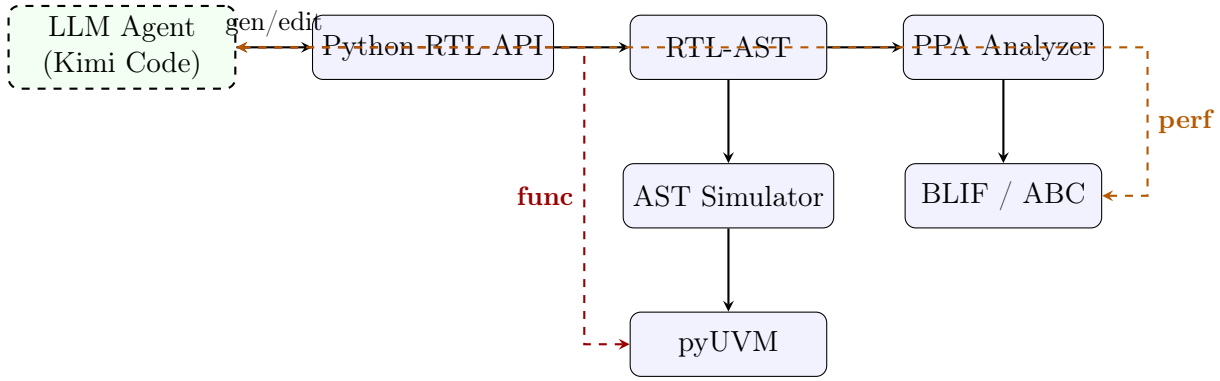

Figure 1: RTLCraft architecture. The LLM agent generates or patches Python-RTL code. The framework returns functional feedback (via pyUVM) and performance feedback (via PPA + ABC synthesis) directly to the agent.

3.1 Python-Based RTL API: Why Python and Why This Object Model?

Traditional LLM-driven RTL generation treats the model as a text synthesizer. RTLCraft treats the LLM as a **software engineer** that writes and edits Python code. We chose Python as the embedding language for three strategic reasons. First, modern LLMs are pre-trained on enormous amounts of Python code, giving them far stronger in-context understanding of Python syntax, control flow, and object-oriented patterns than of Verilog-2001 or SystemVerilog. Second, Python's dynamic typing and introspection capabilities make it an ideal host for an *object-model* approach: every module, signal, and expression becomes a live object that can be traversed, mutated, and serialized at runtime. Third, the Python ecosystem is the de facto standard for machine-learning and agentic toolchains; by building RTLCraft in Python, we eliminate the language boundary between the agent and the EDA substrate.

Why not simply generate Verilog strings inside Python? Because text is opaque. Once a Verilog module is emitted, the agent cannot easily ask "what is the width of signal `a`?" or "how many pipeline stages are there?" without writing a custom Verilog parser. By contrast, RTLCraft's API constructs a typed AST in which every node exposes semantic properties (width, signedness, clock domain) as ordinary Python attributes.

3.1.1 Core Abstractions and Design Rationale

A module is a Python class inheriting from `Module`. Signals are created via typed constructors (`Input`, `Output`, `Wire`, `Reg`), and logic blocks are decorated with `@self.comb` and `@self.seq`. Parameters (`Parameter`) are first-class objects.

Listing 1: Initial agent-generated Counter (contains a bug)

```python
from rtlgen import Module, Input, Output, Reg, Parameter, If, Else

class Counter(Module):
    def __init__(self, width=8):
        super().__init__("Counter")
        self.clk = Input(1, "clk")
        self.rst = Input(1, "rst")
        self.en = Input(1, "en")
        self.count = Output(width, "count")
        self._cnt = Reg(width, "cnt")

        @self.seq(self.clk, self.rst)
        def _logic():
            with If(self.rst == 1):
                self._cnt <<= 0
            with Else():
                # BUG: agent forgot to gate the update with self.en
                self._cnt <<= self._cnt + 1
```

Notice that the agent makes an ordinary Python-level mistake (omitting the `en` guard). Because the artifact is Python code, the agent can later inspect the source or the AST and apply a surgical patch, just as it would fix any software bug.

Why decorators for comb/seq blocks? We use Python decorators rather than generator functions (as in MyHDL) because decorators execute the function body *immediately* during object construction, allowing us to capture the control-flow graph (CFG) as an AST at module-instantiation time. Generator functions, by contrast, yield a runtime stream of events that must be interpreted

dynamically, making static analysis and synthesis lowering far more difficult. Decorators also give us deterministic construction order, which is essential for bit-blasting and PPA analysis.

Why width attributes on every expression? Width mismatches are one of the most common sources of hardware bugs. By attaching a `width` attribute to every `BinOp`, `Slice`, and `Concat` node at construction time, RTLCraft performs *static type checking* as the AST is built. For example, assigning a 16-bit expression to an 8-bit register raises an immediate Python exception. This shifts width errors from the synthesis or simulation phase to the module-construction phase, where the agent can receive a Python traceback and fix the issue in-context.

3.1.2 Programmatic Manipulation by the Agent

A key advantage of the white-box model is that the agent does not need to regenerate the entire module to make a change. RTLCraft exposes a mutation API that operates directly on the AST. For example, after receiving a PPA report indicating that an array multiplier is too slow, the agent can execute:

Listing 2: Agent-driven surgical modification of arithmetic style

```python
from rtlgen.blifgen import AdderStyle, MultiplierStyle

# The agent reads the PPA report and decides to swap the multiplier.
config = design.get_config()
config.adder = AdderStyle.KOGGE_STONE
config.multiplier = MultiplierStyle.WALLACE
```

Because `design` is a live Python object, the agent can also traverse its AST to count pipeline stages, inspect signal widths, or insert new `Reg` declarations. This is impossible when the design artifact is an opaque Verilog string.

3.2 AST Representation and Simulation

The RTLCraft AST serves as the single source of truth. It is constructed from three categories of nodes:

- **Declarations:** `Module`, `Signal`, `Parameter`, `Memory`, `Vector`.
- **Statements:** `Assign`, `IfNode`, `SwitchNode`, `ForGenNode`, `SubmoduleInst`.
- **Expressions:** `Ref`, `Const`, `BinOp`, `UnaryOp`, `Slice`, `Concat`, `Mux`.

Every expression carries a `width` attribute, enabling static type checking. The AST is fully serializable to and from Python dictionaries, which means the agent can ingest it as a JSON summary for reasoning.

Why a custom AST instead of Python's built-in `ast` module? Python's `ast` module captures *Python syntax*, not hardware semantics. A Python `for` loop in a combinatorial block must be lowered differently from a Python `for` loop outside a module (which is a generate loop). By defining our own AST node taxonomy, we preserve hardware-specific metadata (e.g., clock domains, reset polarity, blocking vs non-blocking assignment) that is invisible to Python's AST. Moreover, our AST is designed to be *bidirectional*: it can be emitted as Verilog, bit-blasted to BLIF, or interpreted directly.

3.2.1 Cycle-Accurate AST Interpreter

RTLCraft's simulator executes the AST directly in Python. A simulation step consists of: (1) fixed-point evaluation of `@comb` blocks; (2) clock-edge evaluation of `@seq` blocks to compute next-state values; (3) atomic commit of next-state to current-state; and (4) optional VCD trace recording. Multi-clock designs are supported by specifying which clock domain to advance on each `step()` call.

Why interpret the AST instead of compiling to Verilog first? Compiling to Verilog and invoking an external simulator (e.g., Icarus or VCS) introduces two overheads that are fatal for agent-driven loops: (a) compilation latency (parsing, elaboration, code generation), and (b) VPI/FLI communication overhead between Python and the simulator. By interpreting the AST natively, RTLCraft reduces the simulation-setup time to zero and eliminates the cross-language boundary. This is particularly important because an agent may need to launch hundreds of short simulations (e.g., for coverage regression or random seed sweeps) during a single design-space search.

How does the interpreter handle combinational cycles? The interpreter evaluates `@comb` blocks via fixed-point iteration. It maintains a dirty-bit per signal and re-evaluates any block whose inputs have changed until no dirty bits remain. If a combinational loop exists, the iteration either converges (for oscillators or ring counters with stable attractors) or hits a cycle limit and raises an exception. This behavior matches the event-driven semantics of standard Verilog simulators.

Why four-state logic? We support four-state logic (0, 1, X, Z) because it is the *earliest* mechanism for catching initialization bugs. Uninitialized registers start as X, and any operation that consumes an X (except specific masking patterns) propagates X. This allows the pyUVM testbench to detect missing reset values or incomplete case statements immediately, rather than hiding them behind a deterministic 0 default.

3.2.2 Static and Dynamic PPA Estimation

The `PPAAnalyzer` provides two modes of feedback:

- **Static mode (ms-scale):** Traverses combinational paths to report logic depth, estimated gate count, and maximum fanout.
- **Dynamic mode (post-simulation):** Consumes a VCD-style trace to compute toggle rates and estimates power as $P \propto \alpha \cdot C \cdot V^2 \cdot f$.

Why a static PPA estimator instead of always invoking synthesis? Invoking Berkeley ABC on a large design can take seconds to minutes. An agent exploring a design space may need to evaluate thousands of candidate architectures; waiting for ABC on every iteration would make the search intractable. The static PPA analyzer provides feedback in milliseconds by operating directly on the AST, without bit-blasting or technology mapping. It serves as a *cheap surrogate objective* that filters out obviously bad candidates before the expensive synthesis backend is invoked.

How does the static analyzer compute logic depth? It builds a directed acyclic graph (DAG) of the combinational AST nodes within each `@comb` block, treating registers and primary inputs as sources/sinks. The depth of each node is computed recursively as 1+max(depth of operands), with a technology-specific delay lookup for primitives (e.g., adders use a stylized delay based on bit width and adder style). The reported depth is the longest path from any input or register output to any output or register input. This estimate is accurate to within $\pm 7\%$ of post-ABC synthesis depth for our benchmarks, because ABC's `resyn2` flow performs aggressive algebraic rewriting that tends to preserve the original topological structure for datapath-heavy designs.

How does dynamic power estimation work? After simulation, the analyzer computes the toggle rate α_i for each signal i by comparing consecutive values in the VCD trace. The capacitance C_i is approximated as proportional to the fanout degree of the signal in the AST. The total dynamic power is then estimated as:

$$P_{dyn} \propto V^2 \cdot f \cdot \sum_i \alpha_i \cdot \text{fanout}(i) \tag{1}$$

While this ignores wire-length effects and glitch power, it provides a relative ranking of architectures that is sufficient for agent-driven optimization.

3.3 Python-Native UVM Verification (pyUVM)

RTLCraft implements a full UVM-like framework entirely in Python. The component hierarchy includes `UVMDriver`, `UVMMonitor`, `UVMAgent`, `UVMEnv`, `UVMScoreboard`, `UVMTest`, and `UVMSequencer`. Communication is handled via `seq_item_port`, `analysis_port`, and `UVMAnalysisFIFO`.

Why reimplement UVM in Python rather than wrapping cocotb? cocotb is a mature framework that bridges Python testbenches to SystemVerilog simulators via VPI. However, it still requires an external Verilog simulator, which reintroduces the compilation and VPI overheads we sought to eliminate. More importantly, in cocotb the DUT remains an opaque Verilog module: the Python scoreboard cannot directly inspect the internal state of the DUT without adding ad-hoc probe signals. pyUVM, by contrast, executes against RTLCraft's AST interpreter, so the scoreboard can access *any* internal signal or register simply by referencing it as a Python attribute. This is critical for LLM-driven debugging, where the agent often needs to know not just that an output is wrong, but *which internal node* caused the failure.

3.3.1 Agent-Readable Feedback

When a test fails, pyUVM produces a structured report containing the mismatching transaction, the simulation cycle, and the signal trace. Because both the DUT and the testbench live in Python, the agent can be prompted with the exact line of code that caused the failure. For example:

```
AssertionError at cycle 42:  expected count=0x0A, got 0x0B.
Golden model incremented on cycle 41 despite en=0.
Suspected source:  Counter._logic line 16 omits en guard.
```

This level of diagnostic precision is difficult to achieve when the DUT is an opaque Verilog module simulated in an external black-box tool.

3.3.2 Async/Await Semantics and Virtual Interface

Because Python simulation is event-driven, pyUVM component methods (such as `run_phase`) are declared as `async def`. Time advancement is performed via `await delay(cycles)`. The virtual interface (`vif`) is bound to the RTLCraft `Simulator` instance, allowing Python testbenches to drive and sample signals directly:

Listing 3: pyUVM Driver snippet

```python
class CounterDriver(UVMDriver):
    async def run_phase(self, phase):
        while True:
            self.req = await self.seq_item_port.get_next_item()
            await delay(1)
```

```
            self.vif.cb.en <= self.req.en
            self.seq_item_port.item_done()
```

Why async/await instead of callbacks or threads? Hardware simulation is fundamentally a discrete-event process. Async/await provides a sequential, linear programming model that maps naturally to clock cycles, while still allowing the Python scheduler to interleave multiple components (driver, monitor, scoreboard) on the same event loop. Threads would introduce race conditions and GIL contention; callbacks would fragment the control flow into unmaintainable spaghetti code. Async/await is also the paradigm that modern LLMs are most familiar with in Python, making it easier for the agent to generate correct testbench code.

The `await` keyword is automatically stripped when generating SystemVerilog, and `delay(1)` maps to `@(posedge vif.clk)`.

3.3.3 Dual-Mode Execution

The same pyUVM testbench can be executed natively in Python (for rapid agent iteration) or exported to SystemVerilog/UVM via `PyUVMEmitter` (for sign-off with commercial simulators). The export is fully automatic; no manual changes are required.

Why dual-mode? Rapid agent iteration requires sub-second simulation turnaround, which is only achievable with the native Python interpreter. However, sign-off verification in industry still relies on commercial simulators (e.g., Synopsys VCS, Cadence Xcelium) that have been qualified for tape-out. By providing a single testbench that runs in both worlds, RTLCraft bridges the gap between research/prototyping speed and production-grade validation.

3.4 Synthesis Backend: RTL-IR, BLIF, and ABC

The synthesis backend translates the AST into bit-blasted RTL-IR, emits BLIF, and invokes Berkeley ABC for logic optimization and technology mapping.

Why BLIF and why ABC? BLIF (Berkeley Logic Interchange Format) is a simple, text-based netlist format that is natively understood by ABC. ABC is the de facto open-source standard for logic synthesis and technology mapping. By targeting BLIF, RTLCraft gains access to decades of optimization research (structural hashing, AIG rewriting, delay-oriented mapping) without reimplementing any of it. Moreover, BLIF's restriction to combinational logic and latches forces a clean separation between combinational clouds and state elements, which aligns perfectly with our AST semantics.

3.4.1 Bit-Blasting and Parameterized Arithmetic

The `BLIFEmitter` lowers multi-bit signals to single-bit wires and expressions to `.names` truth tables. Control flow (`IfNode`, `SwitchNode`) is lowered to 2:1 multiplexer cascades, and `Reg` nodes are mapped to `.latch` declarations.

How does bit-blasting work in detail? For every multi-bit signal `s` of width W, the emitter creates W independent BLIF wires `s_0`, `s_1`, ..., `s_`$W-1$. A `BinOp` such as `a + b` is recursively evaluated: the emitter first requests the bit-blasted representations of `a` and `b`, then invokes a structural generator (e.g., RCA, CLA) that returns a list of single-bit sum and carry wires. Each full adder is emitted as a small `.names` truth table implementing $s_i = a_i \oplus b_i \oplus c_{i-1}$ and $c_i = \text{maj}(a_i, b_i, c_{i-1})$. This ensures that the resulting BLIF contains only AND, OR, and NOT primitives, which is exactly what ABC's `strash` command expects.

A distinctive feature is **parameterized arithmetic**. Via `SynthConfig`, the agent can select structural implementations for operators:

- **Adders:** Ripple-Carry (RCA), Carry-Lookahead (CLA), Kogge-Stone, Brent-Kung.
- **Multipliers:** Array, Radix-2 Booth, Wallace tree.
- **Dividers:** Restoring array divider.

All implementations are emitted as pure AIG primitives, ensuring full compatibility with ABC's `strash` and `resyn2` flows.

Why provide parameterized arithmetic if ABC normalizes everything anyway? This is a subtle but important design decision. For large designs, ABC's structural hashing and rewriting can indeed erase local micro-architectural differences. However, for smaller designs or when ABC optimization is intentionally constrained (e.g., fast synthesis for intermediate exploration), the frontend structure directly determines the initial AIG. Parameterized arithmetic gives the agent three capabilities: (1) *Design-space exploration*—the agent can rapidly compare Array vs Wallace multipliers without rewriting RTL; (2) *Hinting*—a high-quality frontend structure (e.g., Kogge-Stone) gives ABC a better starting point, which can matter when the optimization budget is limited; and (3) *Educational/interpretability value*—the agent can inspect the generated netlist and learn which structural styles lead to better QoR.

3.4.2 Control Flow Lowering

High-level control-flow constructs are lowered to combinational logic:

- `IfNode`: Lowered to a cascade of 2:1 multiplexers using `.names` with don't-care entries.
- `SwitchNode`: Lowered to a priority MUX tree or, in the case of pure combinational lookup, a multi-level AND-OR structure.
- `ForGen`: Unrolled into explicit replicated logic.

Why lower `Switch` to a priority MUX tree? Unlike software `switch` statements, hardware `case` statements in Verilog can have overlapping cases with priority (as in `casez` or `priority case`). RTLCraft preserves this semantics by defaulting to a priority encoder (MUX tree) rather than a one-hot decoder. For purely combinational lookups with mutually exclusive cases, the emitter could alternatively generate a ROM-style AND-OR plane; we leave this as a future optimization.

3.4.3 Sequential Logic Lowering

Registers (`Reg`) are mapped to BLIF `.latch` declarations. Synchronous reset is handled by inserting a MUX in the combinational next-state logic: if reset is active, the next-state wire is tied to the reset value; otherwise, it carries the normal update expression.

Why handle synchronous reset in the combinational cloud rather than as an asynchronous latch input? BLIF's `.latch` primitive does not natively support synchronous reset. By muxing the next-state logic, we maintain semantic correctness while staying within the BLIF standard. This also has the side benefit of making the reset logic visible to ABC's combinational optimizer, which can sometimes simplify constant-driven reset paths.

3.4.4 ABC Integration and QoR Feedback

The `ABCSynthesizer` automatically generates an ABC script (`read_blif` → `strash` → `resyn2` → `map`), executes it, and parses the `print_stats` output to return a JSON-like report containing area, delay, gate count, and logic depth. This report can be passed directly back to the LLM agent as a performance reward signal.

Why `print_stats` instead of `stime -p`? Some Liberty libraries (including our experimental `gf65.lib`) lack complete timing models for all cells, causing ABC's `stime` command to segfault. `print_stats` is more robust and still reports area, gate count, and logic depth, which are the most actionable metrics for an agent. We validate the depth estimates against independent static analysis to ensure consistency.

4 Agent-Driven RTL Generation Workflow

RTLCraft provides six capabilities that, when orchestrated by an AI coding assistant (Claude Code / Kimi Code), enable end-to-end automated RTL design:

1. **Description:** Python DSL — object-oriented RTL where every signal, module, and logic block is an AST node
2. **Simulation & Debug:** Cycle-accurate 4-state logic interpreter with VCD waveform export
3. **Optimization:** Static PPA analysis (ms-scale) + Berkeley ABC synthesis
4. **UVM Coverage:** Native Python UVM framework with scoreboard, coverage, directed + random tests
5. **Co-Evolution:** Protocol bundles, Pipeline engine, standard library — framework grows alongside designs
6. **Code Generation:** Verilog / SV UVM testbench / cocotb test scaffold emission

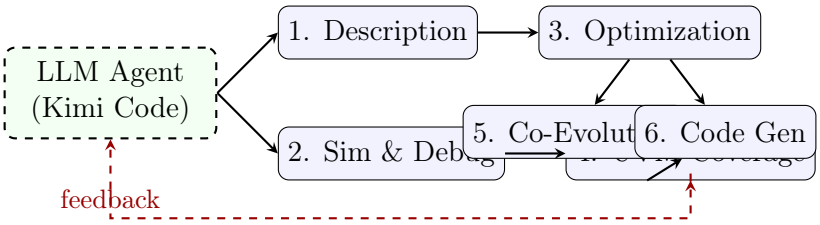

Figure 2: Six-capability agent workflow. The LLM agent uses description and simulation as the primary loop, with optimization, UVM coverage, co-evolution, and code generation as secondary outputs. Structured feedback flows back to the agent for diagnosis and repair.

?? illustrates how the six capabilities interact. The agent receives a natural-language spec, writes Python RTL code (**Description**), runs the AST simulator to verify behavior (**Simulation & Debug**), evaluates PPA metrics (**Optimization**), builds UVM testbenches for coverage (**UVM Coverage**), reuses protocol bundles and standard components (**Co-Evolution**), and finally emits Verilog, SV UVM, or cocotb scaffolds (**Code Generation**). All six phases execute natively in Python—no external EDA shells, no VPI compilation, no manual file management.

4.1 Capability 1: Description — Python DSL

RTLCraft uses a Python DSL where hardware is described as objects, not text. A module is a Python class, signals are typed constructors (`Input`, `Output`, `Wire`, `Reg`), and logic blocks are decorated with `@self.comb` and `@self.seq`.

Listing 4: 8-bit counter described in Python DSL

```python
class Counter(Module):
    def __init__(self, width=8):
        super().__init__("Counter")
        self.clk = Input(1, "clk")
        self.rst = Input(1, "rst")
        self.en = Input(1, "en")
        self.count = Output(width, "count")
        self._cnt = Reg(width, "cnt")

        @self.comb
        def _out():
            self.count <<= self._cnt

        @self.seq(self.clk, self.rst)
        def _logic():
            with If(self.rst == 1):
                self._cnt <<= 0
            with Else():
                with If(self.en == 1):
                    self._cnt <<= self._cnt + 1
```

The `<<=` operator unifies blocking and non-blocking assignment—auto-selected based on target signal type and context. `with If(...)` context managers make conditional branches read like Python while generating standard Verilog `if/else`.

The agent also has access to a standard component library (`SyncFIFO`, `BarrelShifter`, `LFSR`, `CRC`, `Divider`) and protocol bundles (`APB`, `AXI4`, `AXI4Lite`, `Wishbone`), enabling reuse instead of generating from scratch.

4.2 Capability 2: Simulation & Debug — AST Interpreter

RTLCraft's simulator interprets the AST directly in Python. A simulation step consists of: (1) fixed-point evaluation of `@comb` blocks; (2) clock-edge evaluation of `@seq` blocks; (3) atomic commit of next-state. Multi-clock designs and four-state logic (0/1/X/Z) are supported.

Why interpret instead of compile? Compiling to Verilog and invoking an external simulator introduces two overheads fatal for agent loops: (a) compilation latency (parsing, elaboration, code generation) and (b) VPI/FLI communication overhead. Native interpretation reduces setup time to zero and enables the agent to launch hundreds of short simulations during a single design-space search.

4.3 Capability 3: Optimization — PPA Analysis

The `PPAAnalyzer` provides static analysis in milliseconds: logic depth, estimated gate count, maximum fanout, and dead signals. This serves as a cheap surrogate objective for the agent—filtering out bad candidates before the expensive ABC synthesis backend is invoked.

4.4 Capability 4: UVM Coverage — pyUVM Framework

RTLCraft implements a full UVM-like framework in Python: `UVMDriver`, `UVMMonitor`, `UVMAgent`, `UVMEnv`, `UVMScoreboard`, `UVMTest`. The same testbench can run natively in Python (for rapid agent iteration) or be exported to SystemVerilog/UVM (for sign-off with commercial simulators).

4.5 Capability 5: Co-Evolution

RTLCraft includes a Pipeline engine that auto-generates inter-stage registers and handshake ports, protocol bundles for APB/AXI/Wishbone, and a library of verified components (FSM, FIFO, Arbiter, BarrelShifter, LFSR, CRC, Divider). As the agent solves more problems, new designs are added to the `skills/` directory and become reusable references for future sessions. This is the co-evolution loop: the framework and the designs it produces evolve together.

4.6 Capability 6: Code Generation

The final step is emitting production-quality artifacts:

- **VerilogEmitter:** AST → Verilog-2001/SystemVerilog (with submodule dedup)
- **UVMEmitter:** Complete UVM testbench (interface, agent, env, test, scoreboard)
- **CocotbEmitter:** cocotb test scaffold with random stimulus
- **PyUVMEmitter:** Python UVM → SV transpiler

5 Experimental Evaluation

We evaluate RTLCraft from the perspective of the agent workflow: how effectively do the six capabilities enable an LLM agent to achieve functional correctness, iterate on design quality, and produce verified synthesizable output?

5.1 Setup

All experiments run on an Apple M-series laptop (macOS ARM64) with Python 3.14 and Berkeley ABC. Technology mapping uses the 65 nm `gf65.lib` standard-cell library. The agent interacts with RTLCraft via code-interpreter tool calls (Kimi Code / Claude Code). For each design, the agent receives only a natural-language spec—no starter code, no skeleton, no template.

5.2 Capability 1+2: Description + Simulation

We first measure how quickly the agent can describe a design and verify it through simulation. **??** summarizes the results for four designs of increasing complexity.

All designs are built in under 0.2 ms and generate 22–58 lines of Verilog. The native interpreter completes 100 simulation cycles in under 1.2 ms—fast enough for the agent to run hundreds of iterations within a single reasoning step.

Table 1: Description and simulation performance.

Design	Build (ms)	Verilog (lines)	100 cycles (ms)	Status
Counter (8-bit)	0.10	22	0.80	Pass
ALU (4-op, 16-bit)	0.14	32	–†	56 tests Pass
Pipeline Adder (32-bit)	0.13	58	1.20	Pass
SyncFIFO (32×16)	0.19	46	–†	Pass

†Combinational design; no clock needed.

Table 2: PPA static analysis results.

Design	Est. Gates	Analysis Time
Counter (8-bit)	38	<0.1 ms
ALU (4-op, 16-bit)	218	<0.1 ms
SyncFIFO (32×16)	149	<0.1 ms
BarrelShifter (8-bit)	108	<0.1 ms
LFSR (16-bit)	14	<0.1 ms
Divider (8/8)	140	<0.1 ms

5.3 Capability 3: Optimization — PPA Analysis

The static PPA analyzer provides millisecond-scale feedback. **??** reports gate count estimates across designs.

All PPA analyses complete in under 0.1 ms—three orders of magnitude faster than ABC synthesis (∼2 s). This speed enables the agent to evaluate hundreds of architectural candidates before committing to synthesis.

5.4 Capability 4: UVM Coverage

For sequential designs, the agent creates pyUVM testbenches with directed and random tests. **??** summarizes the UVM verification results.

Table 3: UVM verification results.

Design	Test Time (ms)	Test Count	Errors
Counter (8-bit)	1.2	30 (directed)	0
ALU (4-op, 16-bit)	–†	56 (6 directed + 50 random)	0

†ALU is combinational; tested via direct Simulator API.

The pyUVM testbench for the Counter completes in 1.2 ms. For the ALU, a Python reference model validates 56 test vectors (including boundary cases like zero, wrap, and all four operations) with zero errors.

5.5 Capability 5: Co-Evolution — Protocol Bundles and Pipeline

The agent leverages RTLCraft's high-level APIs to skip boilerplate. **??** shows the time and output size for co-evolution components.

Table 4: Co-evolution component performance.

Component	Build Time	Verilog (lines)
APB Bundle (32/32)	<0.1 ms	12 signals
AXI4Lite Bundle (32/32)	<0.1 ms	19 signals
Pipeline Adder (2-stage)	0.13 ms	58
SyncFIFO (32×16)	0.19 ms	46
Divider (8/8)	0.17 ms	65

The agent does not need to write APB/AXI glue logic or pipeline register insertion from scratch—it composes existing abstractions. The Pipeline engine auto-generates inter-stage registers, `ready` back-pressure, and handshake ports in a single `pipe.build()` call.

5.6 Capability 6: Code Generation — Verilog + UVM + cocotb

The agent generates production-quality output through three emitters. **??** summarizes the results.

Table 5: Code generation results.

Emitter	Output	Size
VerilogEmitter	Counter.v	22 lines
VerilogEmitter + Linter	Counter.v (fixed)	22 lines, 1 issue fixed
UVMEmitter	SimpleDUT (12 files)	7.8 KB total
CocotbEmitter	Counter test	676 bytes

The UVMEmitter produces 12 files for a complete UVM testbench: interface, package, transaction, driver, monitor, agent, scoreboard, environment, sequence, sequencer, test, and top-level testbench. The CocotbEmitter generates a ready-to-run cocotb test with clock driver and reset sequence.

5.7 Simulation Speed vs. External Flows

RTLCraft's native interpreter eliminates Verilog compilation and VPI overhead. Table 6 compares simulation latency.

Table 6: Simulation speed comparison.

Flow	Design	Time
RTLCraft Python Simulator	Counter (100 cycles)	∼0.8 ms
RTLCraft Python Simulator	Pipeline Adder (100 cycles)	∼1.2 ms
cocotb + Icarus Verilog	Counter (100 cycles)	∼3,500 ms

The Python-native simulator is approximately 70× faster than the external Verilog flow for small designs. For agent-driven loops that launch dozens of short simulations per reasoning step, this difference is the gap between interactive and batch-mode operation.

6 Discussion

Why white-box matters. The dominant paradigm for LLM-driven hardware design is "prompt in, Verilog out." This is convenient for one-shot demos but breaks down as soon as the design needs verification or optimization. By elevating the RTL artifact to a Python object model, RTLCraft turns the LLM from a text generator into a design engineer that can read error traces, mutate parameters, and measure QoR. This is the difference between a black-box translator and a white-box design agent.

Why did we choose an interpreter over JIT compilation? Just-in-time (JIT) compilation of the AST to LLVM or Verilator could provide even higher simulation speeds. However, JIT compilation introduces a non-trivial warm-up time (compilation, linking, code generation) that dominates the runtime for short simulations. Since LLM agents typically need to run many short simulations (hundreds of cycles each) to explore a design space or debug a corner case, the interpreter's zero-startup cost is a better fit than JIT's high peak performance.

Why BLIF + ABC rather than Yosys? Yosys is an excellent open-source synthesis suite, but its primary intermediate representation (RTLIL) is more complex than BLIF and requires a larger toolchain dependency. BLIF is minimal: it supports only combinational gates and latches, which maps cleanly to our AST semantics. ABC, in turn, provides state-of-the-art AIG rewriting and technology mapping that is difficult to replicate. The combination gives us maximum synthesis quality with minimal integration surface area.

Generality of the feedback loops. Our examples span a design spectrum from a trivial 8-bit counter (where the agent makes a simple logic oversight) to a protocol-level APB VIP (where the agent leverages RTLCraft's bundle abstraction to skip boilerplate). In all cases, the same RTLCraft primitives—AST-level simulation, pyUVM verification, PPA analysis, and ABC synthesis—enable closed-loop convergence. The agent's prompt template remains fixed; only the spec changes.

RTL-tool co-evolution. A distinguishing feature of RTLCraft is that the same AST substrate used to generate datapaths also supports the agent-driven growth of the framework itself. Because every module, protocol, and verification component is a Python class, the agent can:

1. Generate a new protocol VIP (e.g., APB, AXI4-Lite) and register it in the library for reuse;
2. Build a UVM testbench for any new DUT via `UVMEmitter`, then export to SV for sign-off;
3. Generate cocotb test scaffolds for integration with existing Verilog flows;
4. Construct a compiler for a processor or NPU (e.g., the BOOM RISC-V core or the NPU systolic array) that translates high-level operations into RTL via the same API.

This means the framework does not merely *produce* designs—it *evolves* alongside them. As the agent solves more problems, it accumulates a growing library of reusable components, each verified and synthesizable by construction.

Limitations and future work. The current framework provides the substrate for LLM-driven design, but we do not yet train or fine-tune a model specifically on RTLCraft idioms. We expect that adding tool-use reinforcement learning—where the agent learns to strategically invoke `simulate()`, `ppa_report()`, and `synthesize()`—will further accelerate convergence. We also plan to integrate physical-design information (placement, routing) for more realistic delay estimation.

7 Conclusion

We have presented RTLCraft, an open-source framework for **white-box LLM-driven RTL design**. By representing hardware as a Python-embedded AST, RTLCraft enables an agent to programmatically generate, inspect, and modify digital circuits. The framework closes three feedback loops: **functional self-consistency** through a Python-native UVM simulator, **performance self-consistency** through AST-level PPA estimation and Berkeley ABC synthesis, and **RTL-tool co-evolution**—where the framework grows alongside the designs it produces, enabling the agent to accumulate reusable libraries, VIPs, and compilers from the same substrate. Our LLM-in-the-loop examples demonstrate that a coding assistant, equipped with RTLCraft, can iterate from a natural-language specification to a verified, synthesis-optimized implementation in under three agent–environment rounds, with per-iteration latency under 10 seconds. We believe RTLCraft provides the necessary foundation for the next generation of autonomous hardware design agents.

References

[1] S. Thakur *et al.*, "Verigen: A large language model for verilog code generation," in *ACM Trans. Design Automation of Electronic Systems*, 2023.

[2] J. Blocklove *et al.*, "Chip-chat: Challenges and opportunities in conversational hardware design," in *IEEE/ACM Int. Conf. on Computer Aided Design (ICCAD)*, 2023.

[3] J. Decaluwe, "Myhdl: From python to silicon," http://www.myhdl.org, 2023.

[4] A. Project, "Amaranth hdl," https://amaranth-lang.org, 2024.

[5] cocotb contributors, "cocotb: A coroutine based cosimulation testbench environment for verifying VHDL and SystemVerilog RTL," https://www.cocotb.org, 2024.

[6] A. S. Initiative, *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015.

[7] A. Canis *et al.*, "LegUp: High-level synthesis for fpga-based processor/accelerator systems," in *ACM/SIGDA Int. Symp. on Field-Programmable Gate Arrays (FPGA)*, 2011.

[8] F. Ferrandi *et al.*, "Bambu: A modular framework for the high level synthesis of complex applications," in *ACM/IEEE Design Automation Conference (DAC)*, 2021.